

Herança, Polimorfismo e Interfaces

Bib: Java: An Introduction to Problem Solving & Programming, Walter Savitch

Cap. 8: Inheritance, Polymorphism and Interfaces

Programação III (PG3)

João Pedro Patriarca, (joao.patriarca@isel.pt),

Gabinete F.0.23 do edifício F, LEETC, ADEETC

Slides inspirados nos slides da professora Manuela Sousa



Resumo

- Herança
- Polimorfismo
- Interfaces

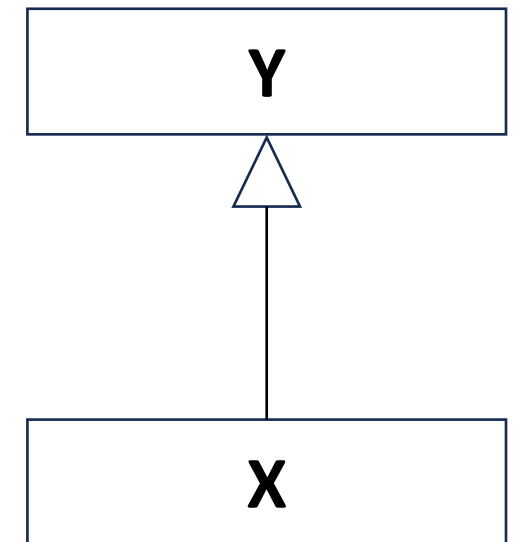
Resumo

- Herança
- Polimorfismo
- Interfaces

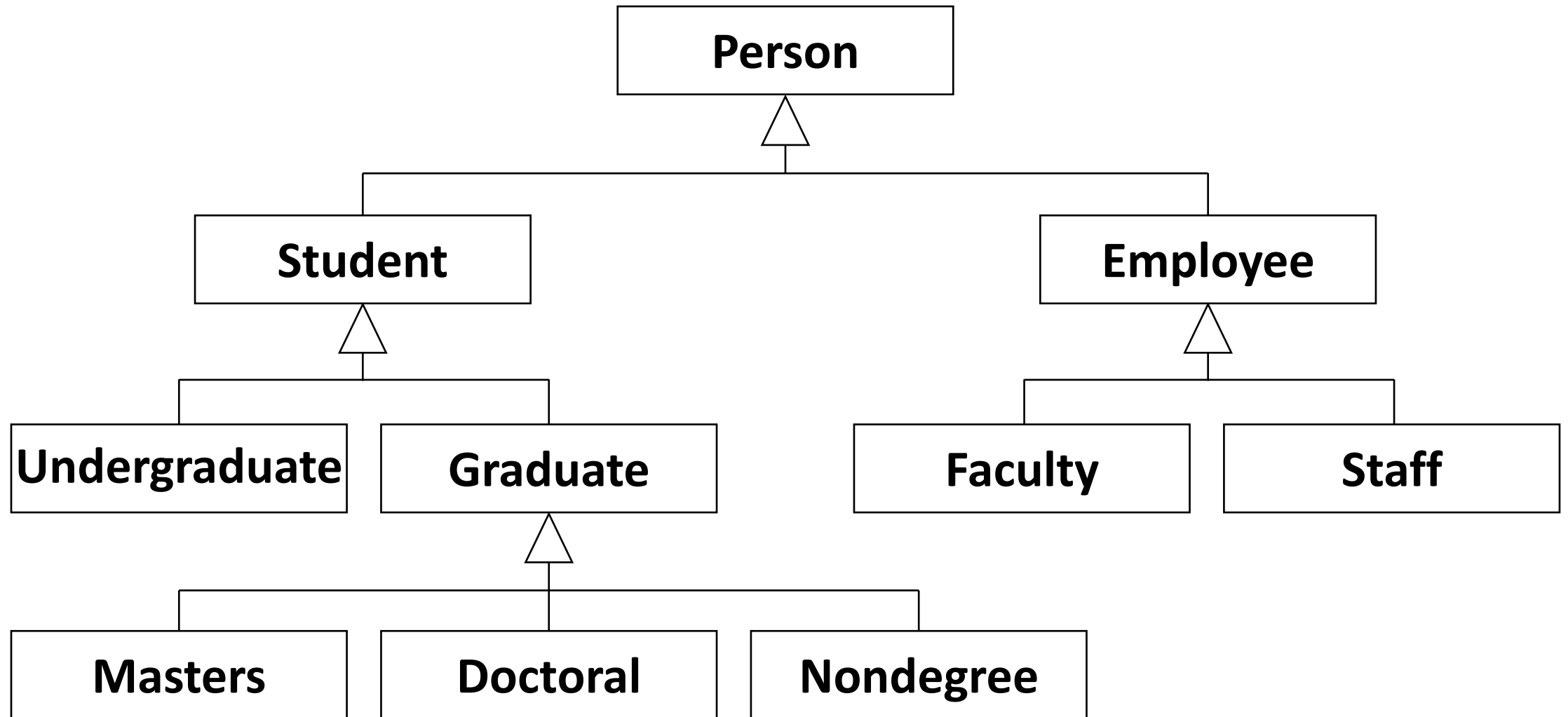
Herança – o que é?

- Suporte à definição de uma *classe especializada* (classe X) de uma *classe base* (classe Y)
- A classe X *herda* variáveis e métodos da classe Y
- A definição da classe X pode adicionar variáveis e métodos ou sobrepor métodos da classe Y
- Diz-se que a classe X *deriva* da classe Y, ou *estende* a classe Y, onde a classe Y é a *superclasse* e a classe X é a *subclasse*
- A relação de herança entre duas classes implementa a relação “é-um”, do inglês “is-a”. Atenção: a relação não é bidirecional. A classe X “é-um” Y mas a classe Y não “é-um” X
- Em Java, uma classe apenas pode derivar de outra, ou seja, não pode estender duas ou mais classes

Relação de herança num diagrama de classes



Exemplo de uma hierarquia de classes (Fig. 8.1, Savitch)



Implementação de Person e Student (List 8.1/2, Savitch)

```
public class Person {  
    private String name;  
    public Person(String name) {  
        this.name = name;  
    }  
  
    public String getName() {  
        return name;  
    }  
  
    public void setName(String newName) {  
        name = newName;  
    }  
}
```

Tratado adiante

```
public class Student extends Person {  
    private int number;  
    public Student(String name, int number) {  
        super(name);    this.number = number;  
    }  
  
    public void reset(String name, int number) {  
        setName(name);    this.number = number;  
    }  
  
    public int getNumber() { return number; }  
    public void setNumber(int number) {  
        this.number = number;  
    }  
}
```

Demo herança

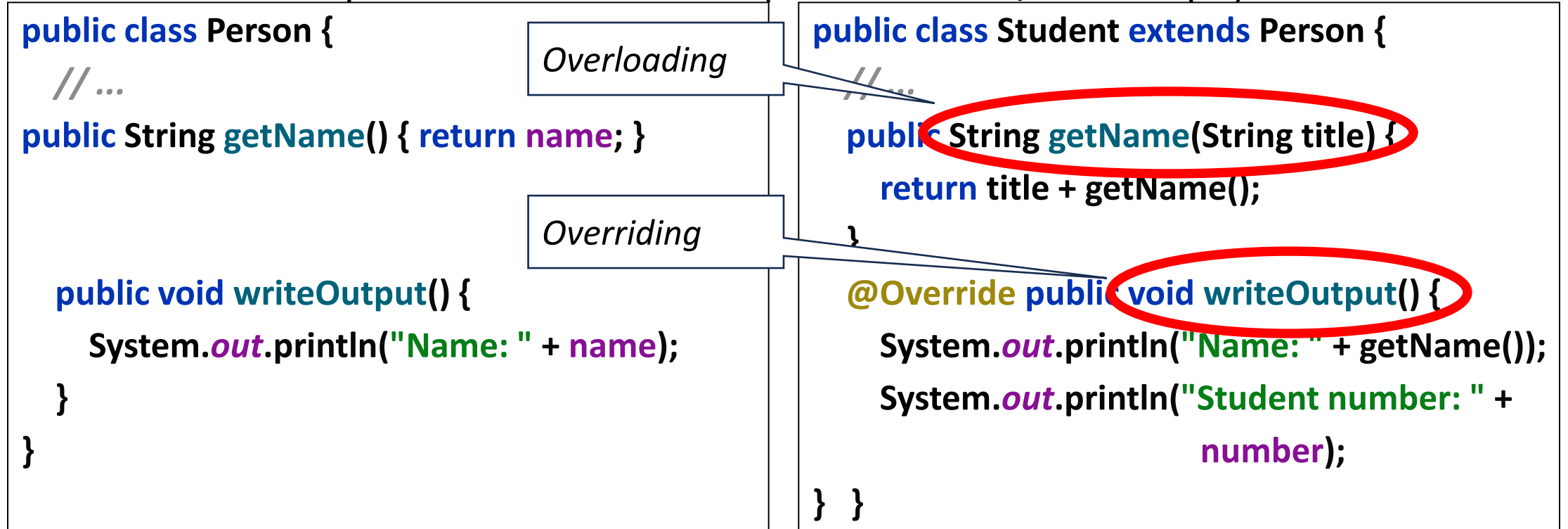
- O objeto Student herda a variável de instância name de Person;
- Herda igualmente os métodos de instância getName e setName
- Student não tem acesso direto à variável name em Person porque está marcada como private mas consegue aceder-lhe através dos métodos herdados (não privados)
- Métodos privados na classe base são igualmente inacessíveis na classe derivada

```
public class InheritanceDemo {  
    public static void main(String[] args) {  
        Student student = new Student(  
            "Joaquim", 52000  
        );  
        student.setName("Patrícia");  
        student.setNumber(52001);  
        System.out.println(  
            student.getNumber() + ", \"\" +  
            student.getName() + \"\"  
        );  
    }  
}
```

A black rectangular box containing the text "52001, \"Patrícia\"" in white, representing the output of the Java program shown in the code block above.

Sobreposição versus sobrecarga de métodos (1 de 2)

- Sobreposição (*overriding*): definição na classe derivada de um método com a mesma assinatura que um método definido na classe base; o método não pode ser privado, nem marcado como `static` ou `final`
- Sobrecarga (*overloading*): definição de dois ou mais métodos com o mesmo nome mas com parâmetros diferentes (em número e/ou em tipo)



Sobreposição versus sobrecarga de métodos (2 de 2)

- A sobreposição (*overriding*) é diferente da sobrecarga (*overloading*)
- A classe Student tem dois métodos getName (*overloaded*) mas apenas tem um método writeOutput (*overridden*)

```
public class OverrideVsOverloadDemo {  
    public static void main(String[] args) {  
        Student student = new Student(  
            "Miguel", 47000  
        );  
        student.writeOutput();  
        System.out.println(student.getName());  
        System.out.println(  
            student.getName("Sr. ");  
        )  
    }  
}
```

```
Name: Miguel  
Student number: 47000  
Miguel  
Sr. Miguel
```

Modificador `final`

- O modificador `final`, aplicado a um método de instância, impossibilita a sobreposição do método em classes derivadas
- O mesmo modificador `final`, aplicado a uma classe, impossibilita que a classe seja estendida
 - Ex: `public final class <classname> { ... }`
- O mesmo modificador `final`, aplicado a uma variável da classe, impossibilita que a variável possa ser alterada durante o seu tempo de vida

Construtores (1 de 4)

- Na criação de um objeto de uma classe especializada, o estado herdado das classes base tem de ser iniciado previamente (promover a execução dos construtores das classes base)*
- A chamada explícita é realizada através da instrução `super([<arguments>]);`
- Na ausência de chamada explícita de um construtor da classe base, é chamado implicitamente o construtor sem parâmetros
 - O construtor sem parâmetros tem de estar definido na classe base

** A partir da versão 25 do Java, é possível introduzir código antes da chamada ao construtor da classe base (JEP 513: Flexible Constructor Bodies); o código prévio não pode nunca fazer referência ao objeto (`this`) porque o objeto ainda não foi construído*

Construtores (2 de 4)

- A chamada do construtor da classe base apenas pode acontecer uma vez e tem de ser realizado antes de qualquer acesso ao estado interno
 - O estado herdado tem de ser iniciado em primeiro – a inicialização do estado especializado pode depender do estado herdado
 - A exceção é com a sobrecarga de construtores na classe derivada: como foi visto, a chamada de um construtor a partir de outro é feito por via da palavra `this` – neste caso, o construtor que chama outro construtor da mesma classe não pode fazer `super`; `this` e `super` não podem aparecer no mesmo construtor, caso contrário seriam executados dois construtores da classe base
- Não é possível efetuar a chamada de um construtor de classes base superior a um nível na relação de herança
 - Ex: `super.super(<args>)` é inválido

Construtores – exemplo (3 de 4)

```
public class ConstructorsA {  
    private int field1;  
    private double field2;  
  
    public ConstructorsA(int f1) {  
        System.out.print(" -> .ctorA (int)");  
        field1 = f1;  
    }  
    public ConstructorsA(int f1, double f2) {  
        System.out.print(" -> .ctorA(int, double)");  
        field1 = f1; field2 = f2;  
    }  
}
```

```
public class ConstructorsB extends ConstructorsA {  
    private int field3;  
  
    public ConstructorsB(int f3, int f1, double f2) {  
        super(f1, f2);  
        System.out.print(" -> .ctorB(int, int, double)");  
        field3 = f3;  
    }  
    public ConstructorsB(int f3, int f1) {  
        this(f3, f1, 1.5);  
        System.out.print(" -> .ctorB (int, int)");  
    }  
    // public ConstructorsB(int f3) {  
    //     field3 = f3;  
    // }  
}
```

Erro de compilação porque a classe base não define construtor sem parâmetros

Construtores – exemplo (4 de 4)

```
public class ConstructorsDemo {  
    public static void main(String[] args) {  
        System.out.println("Create x");  
        ConstructorsB x = new ConstructorsB(10, 20);  
        System.out.println("\nCreate y");  
        ConstructorsB y = new ConstructorsB(100, 200, 5);  
        System.out.println();  
    }  
}
```

Create x

-> .ctorA(int, double) -> .ctorB(int, int, double) -> .ctorB(int, int)

Create y

-> .ctorA(int, double) -> .ctorB(int, int, double)

Chamada de métodos *overridden*

- Na implementação de um método da classe derivada é possível chamar métodos *overridden* da classe base
 - Ao contrário dos construtores, a chamada não está limitada a um método específico da classe base, pode ser realizada em qualquer ponto da implementação e pode ocorrer uma ou mais vezes

```
public class Student extends Person {  
    // ...  
    @Override public void writeOutput() {  
        System.out.println("Name: " + getName());  
        System.out.println("Student number: " +  
                             number);  
    }  
}
```

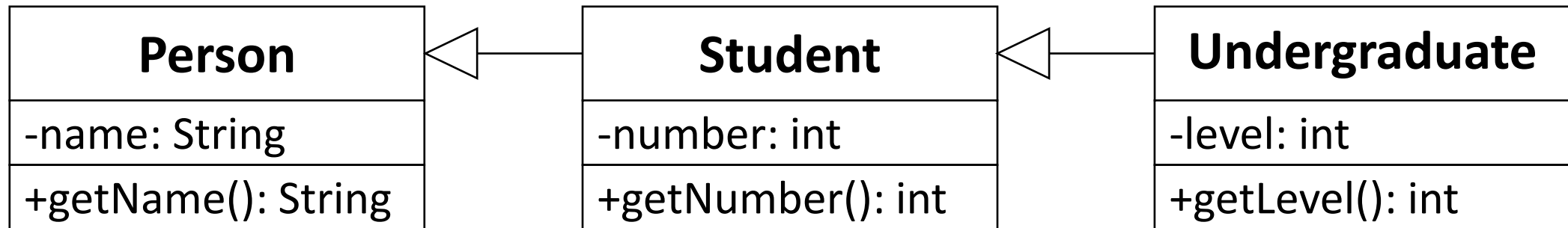
Implementado em Person.
Preferível solução ao lado

```
public class Student extends Person {  
    // ...  
    @Override public void writeOutput() {  
        super.writeOutput();  
        System.out.println("Student number: " +  
                             number);  
    }  
}
```

- Tal como na construção de um objeto, não é possível chamar métodos *overridden* das classes base superior a um nível na relação de herança

Compatibilidade de tipos (1 de 2)

- Considerar hierarquia de classes



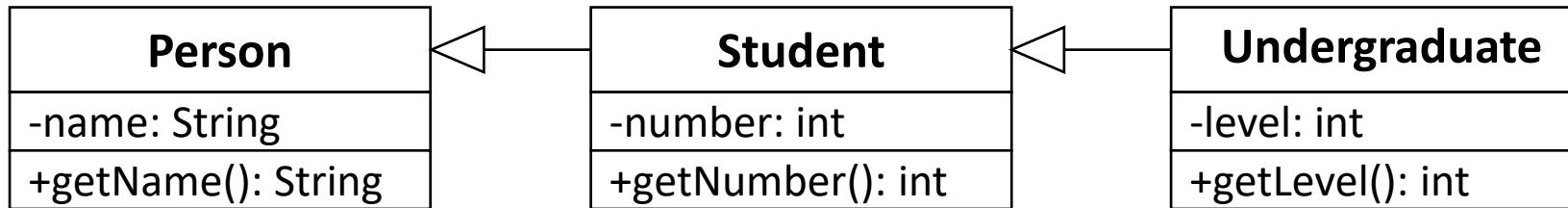
- Undergraduate “is-a” Student e Student “is-a” Person, logo Undergraduate “is-a” Person; o contrário não se verifica

Como Undergraduate é Student, é válido uma variável do tipo Student guardar referência para objeto do tipo Undergraduate

```
public class TypesCompatibilityDemo {
    public static boolean compareNumbers(Student s1, Student s2) { /* ... */ }
    public static void main(String[] args) {
        Student s1 = new Student("A", 52000); Undergraduate u1 = new Undergraduate("B", 52000, 2);
        boolean hasSameNumber = compareNumbers(s1, u1);
        System.out.println((hasSameNumber ? "Same": "Different") + " number");
    } }
```

Same number

Compatibilidade de tipos (2 de 2)

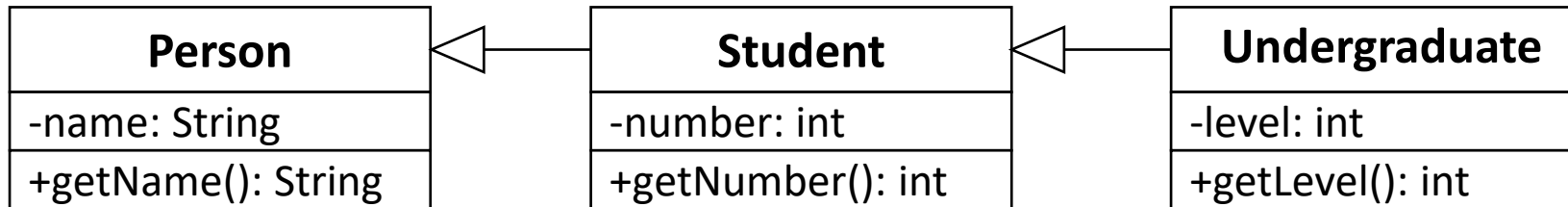


```
public class TypesCompatibilityDemo {
    public static void main(String[] args) {
        Person p1 = new Student("Ana", 52000); Person p2 = new Undergraduate("Bruno", 52001, 2);
        //p1.getNumber(); // Erro de compilação: o método getNumber não existe na classe Person
        System.out.println("Name: " + p2.getName()); // Escreve no stdout: Name: Bruno
        //Student s2 = new Person("Zé"); // Erro de compilação: Person não é um Student
        //Undergraduate u2 = (Undergraduate)p1; // Erro de execução: exceção ClassCastException
        Undergraduate u3 = (Undergraduate)p2; // p2 referencia objeto Undergraduate (válido)
        System.out.println("Level: " + u3.getLevel()); // Escreve no stdout: Level: 2
    }
}
```

Como se determina se é viável fazer cast ou não?

Compatibilidade de tipos – operador `instanceof`

- O operador `instanceof` verifica se o objeto é de um determinado tipo de classe

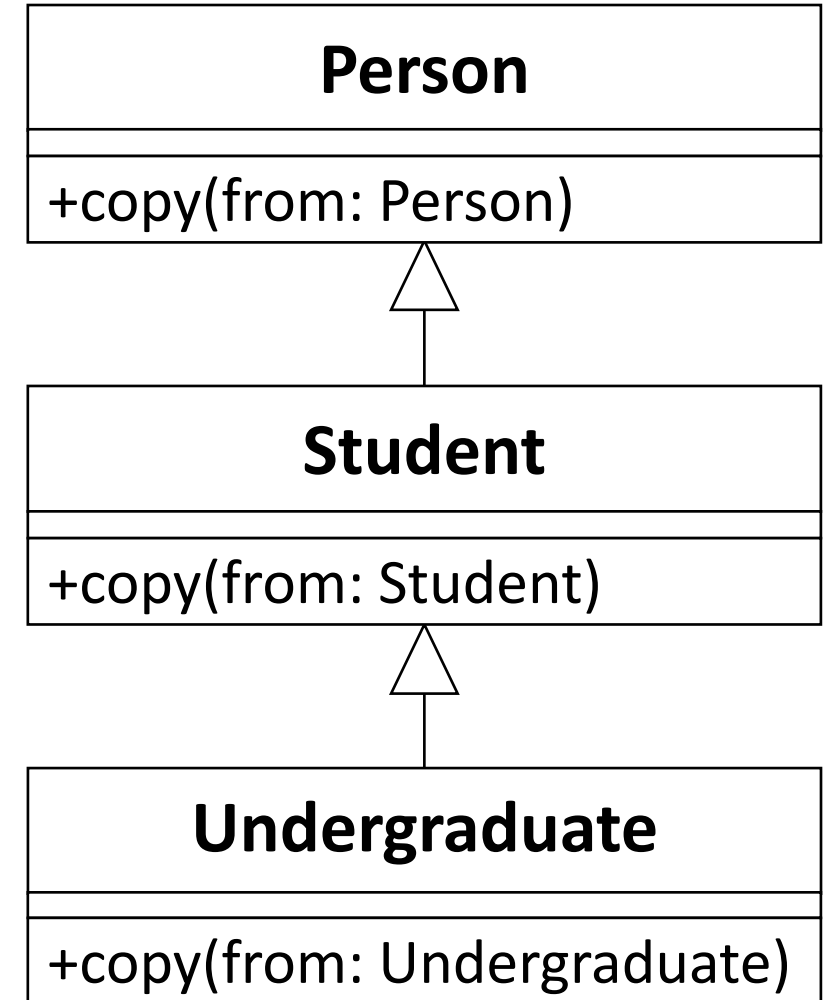


```
public class TypesCompatibilityDemo {
    public static void main(String[] args) {
        Person p3 = new Student("Carla", 52002);
        boolean b1 = p3 instanceof Student;
        boolean b2 = p3 instanceof Undergraduate;
        boolean b3 = p3 instanceof Person;
        System.out.println("b1: " + b1 + ", b2: " + b2 + ", b3: " + b3);
    } }
```

b1: true, b2: false, b3: true

Sobrecarga versus sobreposição revisitado (1 de 5)

- Os métodos `copy` em cada classe estão sobrepostos ou estão sobrecarregados?
- Segundo a definição
 - Sobrecarga: métodos com nomes iguais mas parâmetros diferentes
 - Sobreposição: métodos com nomes e parâmetros iguais
- A dúvida pode manter-se uma vez que Undergraduate é um (*is-a*) Student e Student é um (*is-a*) Person

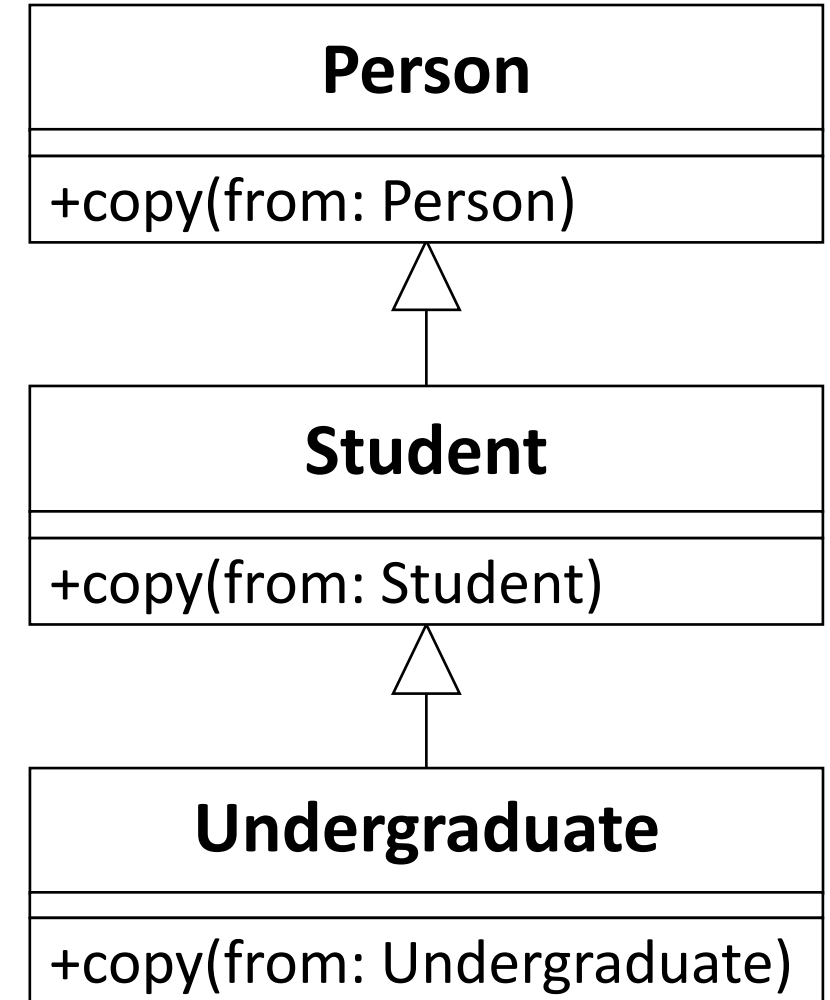


Sobrecarga versus sobreposição revisitado (2 de 5)

```
public class Person {  
    public void copy( Person from ) {  
        System.out.println( "\tPerson copy" );  
        name = from.name;  
    }  
}
```

```
public class Student extends Person {  
    public void copy( Student from ) {  
        System.out.println( "\tStudent copy" );  
        copy( (Person)from ); number = from.number;  
    }  
}
```

```
public class Undergraduate extends Student {  
    public void copy ( Undergraduate from ) {  
        System.out.println( "\tUndergraduate copy" );  
        copy( (Student)from ); level = from.level;  
    }  
}
```



Sobrecarga versus sobreposição revisitado (3 de 5)

```
public class OverrideVsOverloadDemo2 {  
    public static void main( String[] args ) {  
        Undergraduate u1 = new Undergraduate( "Maria", 52000, 1 );  
        Undergraduate u2 = new Undergraduate( "Tomé", 52001, 2 );  
        System.out.println( "Copy as U" );  
        u1.copy( u2 );  
        System.out.println( "Copy as S" );  
        u1.copy( (Student)u2 );  
        System.out.println( "Copy as P" );  
        u1.copy( (Person)u2 );  
    }  
}
```

```
Copy as U  
    Undergraduate copy  
    Student copy  
    Person copy  
Copy as S  
    Student copy  
    Person copy  
Copy as P  
    Person copy
```

O resultado da execução permite perceber que existem três métodos copy na classe Undergraduate, portanto está sobrecarregado

O compilador seleciona o método a executar função do tipo do argumento

Sobrecarga versus sobreposição revisitado (4 de 5)

```
public class Person {  
    @Override public void copyOverride( Object from ) {  
        if ( !(from instanceof Person p) ) return;  
        System.out.println("\tPerson copy");    name = p.name;  
    } }  

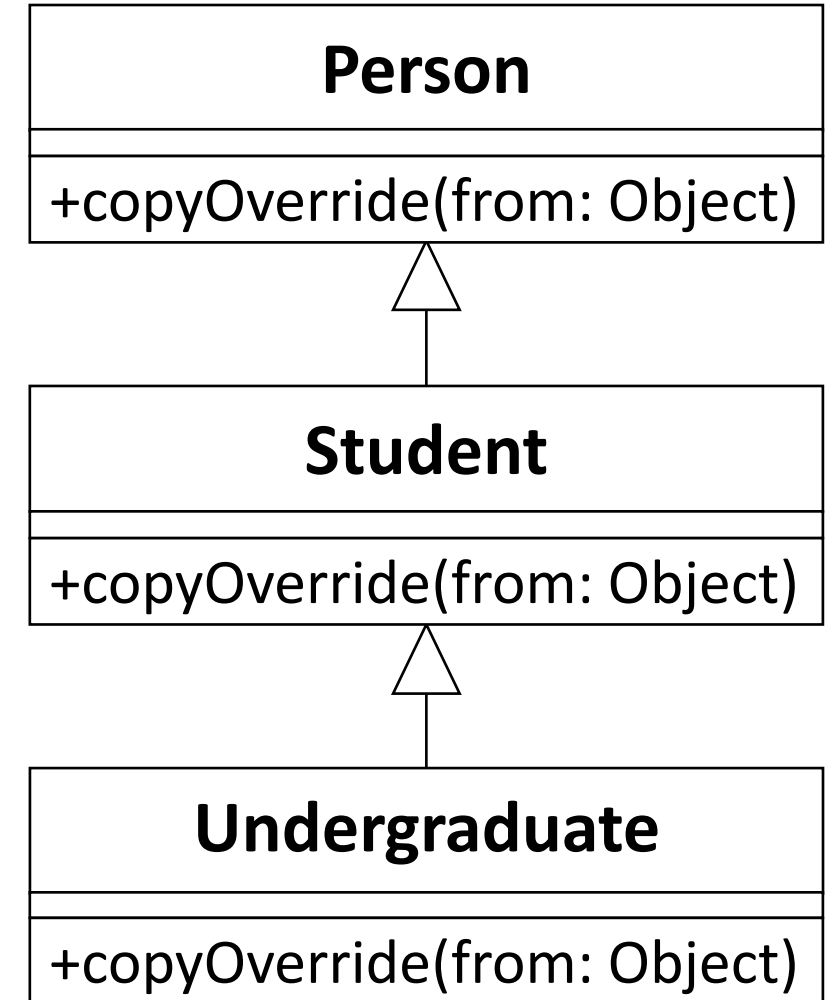
```

```
public class Student extends Person {  
    @Override public void copyOverride( Object from ) {  
        if ( !(from instanceof Student s) ) return;  
        System.out.println("\tStudent copy");  
        super.copyOverride(from);    number = s.number;  
    } }  

```

```
public class Undergraduate extends Student {  
    @Override public void copyOverride( Object from ) {  
        if ( !(from instanceof Undergraduate u) ) return;  
        System.out.println("\tUndergraduate copy");  
        super.copyOverride(from);    level = u.level;  
    } }  

```



Sobrecarga versus sobreposição revisitado (5 de 5)

```
public class OverrideVsOverloadDemo2 {  
    public static void main( String[] args ) {  
        Undergraduate u1 = new Undergraduate( "Maria", 52000, 1 );  
        Undergraduate u2 = new Undergraduate( "Tomé", 52001, 2 );  
        System.out.println( "Override copy as U" );  
        u1.copyOverride( u2 );  
        System.out.println( "Override copy as S" );  
        u1.copyOverride( (Student)u2 );  
        System.out.println( "Override copy as P" );  
        u1.copyOverride( (Person)u2 );  
    }  
}
```

```
Override copy as U  
    Undergraduate copy  
    Student copy  
    Person copy  
Override copy as S  
    Undergraduate copy  
    Student copy  
    Person copy  
Override copy as P  
    Undergraduate copy  
    Student copy  
    Person copy
```

O resultado da execução permite perceber que existe um único método `copyOverride` na classe `Undergraduate`, portanto está sobreposto

Classe Object

- Todos os tipos classe têm como classe base raiz a classe Object

- Tipos classe devem sobrepor, por norma, os métodos de Object

`public String toString()`

`public boolean equals(Object obj)`

- Atenção que na definição do método `equals`, o tipo do parâmetro é `Object` e não do tipo da classe especializada para que o método seja sobreposto e não sobrecarregado

Sobreposição

```
public class Undergraduate extends Student {  
    public boolean equals( Object o ) {  
        if ( !(o instanceof Undergraduate that) )  
            return false;  
        return super.equals(o) && level==that.level;  
    }  
}
```

Sobrecarrega

```
public class Undergraduate extends Student {  
    public boolean equals( Undergraduate o ) {  
        if ( !(o instanceof Undergraduate that) )  
            return false;  
        return super.equals(o) && level==that.level;  
    }  
}
```

Resumo

- Herança
- Polimorfismo
- Interfaces

Polimorfismo – o que é?

- Polimorfismo deriva do Grego e significa “*muitas formas*”
- Considerando o exemplo
 - As classes Student e Undergraduate sobrepõem o método writeOutput

```
public class Person {  
    public void writeOutput() {  
        System.out.println("Name: " + name);  
    }  
}
```

```
public class Student extends Person {  
    @Override public void writeOutput() {  
        super.writeOutput();  
        System.out.println("Number: " + number);  
    }  
}
```

```
public class Undergraduate extends Student {  
    @Override public void writeOutput() {  
        super.writeOutput();  
        System.out.println("Level: " + level);  
    }  
}
```

Polimorfismo – qual o resultado? (List. 8.6, Savith)

```
public class PolymorphismDemo {  
    public static void main( String[] args ) {  
        Person [] people = new Person[3];  
        people[0] = new Undergraduate( "Cotty, Manny", 4910, 1 );  
        people[1] = new Student( "DeBanque, Robin", 8812 );  
        people[2] = new Person( "Bugg, June" );  
        for ( Person p: people ) {  
            p.writeOutput();  
            System.out.println();  
        }  
    }  
}
```

Polimorfismo – resultado

```
public class PolymorphismDemo {  
    public static void main( String[] args ) {  
        Person [] people = new Person[3];  
        people[0] = new Undergraduate( "Cotty, Manny", 4910, 1 );  
        people[1] = new Student( "DeBanque, Robin", 8812 );  
        people[2] = new Person( "Bugg, June" );  
        for ( Person p: people ) {  
            p.writeOutput();  
            System.out.println();  
        }  
    }  
}
```

```
Name: Cotty, Manny  
Student number: 4910  
Undergraduate level: 1  
  
Name: DeBanque, Robin  
Student number: 8812  
  
Name: Bugg, June
```

O resultado demonstra que é o tipo do objeto que determina qual o método `writeOutput` executado

Ligação dinâmica (*dynamic binding*) ou ligação tardia (*late binding*)

- O método a executar é determinado em tempo de execução do programa e depende do tipo do objeto e não do tipo da variável que armazena a referência para o objeto
- Permite que o mesmo programa, sem quaisquer alterações, execute código que ainda não foi criado. As condições são:
 - Que seja criada uma classe dentro da hierarquia de herança já existente (e.g. Graduate, Employee, Staff)
 - Que a classe criada sobreponha o método em causa (writeOutput)
- Em Java, todos os métodos de instância são virtuais, por omissão, e, portanto, passíveis de serem sobrepostos e envolvidos em ligação dinâmica; noutras linguagens tem de se indicar explicitamente
 - O mecanismo de ligação dinâmica envolve penalização no tempo de execução
 - No desenho de uma classe, métodos que não sejam passíveis de sobreposição devem ser marcados como `final`

Resumo

- Herança
- Polimorfismo
- Interfaces

Classe abstrata

- Objetivo de classes abstratas

Tipos base servem para reunir características e operações comuns a vários tipos especializados; por vezes, existem operações cuja implementação depende dos tipos especializados e, como tal, devem ser marcadas como métodos abstratos na classe base e exigir que sejam as classes especializadas a fornecerem as suas implementações

- Uma classe abstrata impede que sejam criados objetos da classe

```
public abstract <ClassName> { ... }
```

- A definição de um método da classe como abstrato implica que a respetiva classe seja igualmente abstrata

- Na mesma classe pode existir mais do que um método abstrato

- Um método abstrato não tem código

```
public abstract <type> <methodName> ([parameters]);
```

- Uma classe derivada, não abstrata, tem de sobrepor e fornecer uma implementação dos métodos abstratos para poder criar objetos da classe derivada

Interface

- É como se fosse uma classe abstrata constituída apenas por métodos abstratos e constantes
 - Não inclui campos de instância, construtores nem definições de métodos*¹
 - Pode incluir a definição de constantes (`public`, `final` e `static`) e de métodos abstratos*¹ públicos (dada a imposição, pode omitir-se a visibilidade e modificadores)
- Conceptualmente, separa a interface pública de uma classe da respetiva implementação

```
public interface <InterfaceName> { ... }
```
- Uma interface pode estender outra interface

```
public interface <InterfaceName> extends <InterfaceBase> { ... }
```
- Convenções de nomes iguais à de uma classe

*¹ a partir da versão 8 do Java é dado suporte à definição de métodos com implementações por omissão (*default methods*) e à definição de métodos estáticos

Interface – utilização

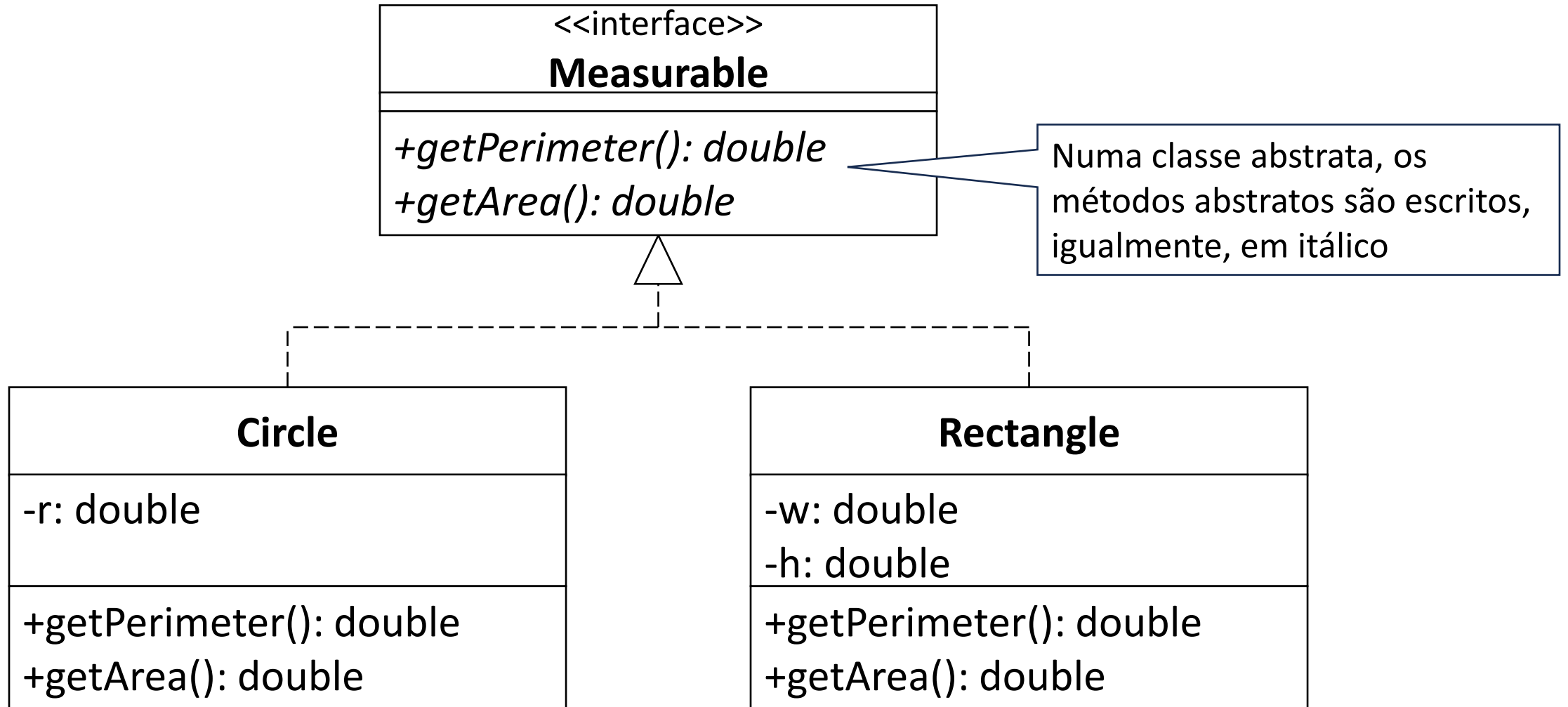
- Uma classe pode implementar uma ou mais interfaces

```
public class <ClassName> implements <InterfaceName1>,  
    <InterfaceName2>, ... { /* body */ }
```
- Diferentes classes podem implementar a mesma interface
- Para poder instanciar um objeto da classe, a classe tem de implementar todos os métodos abstratos da interface
 - Se não implementar algum método da interface, a classe tem de ser marcada como abstrata
- A referência para o objeto da classe pode ser armazenada numa variável do tipo da interface

```
public classe X implements A, B, C { ... }  
A ax = new X(); B bx = new X(); C cx = new X();
```

 - As variáveis ax, bx, e cx estão condicionadas à utilização dos métodos definidos nas respectivas interfaces A, B e C

Interface – exemplo – UML



Interface – exemplo – implementação

```
public interface Measurable { double getPerimeter(); double getArea(); }
```

```
public class Circle implements Measurable {  
    private double r;  
    public Circle(double radius) { r = radius; }  
    @Override public double getPerimeter() { return 2 * Math.PI * r; }  
    @Override public double getArea() { return Math.PI * r * r; }  
    @Override public String toString() { return "r=" + r; }  
}
```

```
public class Rectangle implements Measurable {  
    private double w, h;  
    public Rectangle(double width, double height) { this.w = width; this.h = height; }  
    @Override public double getPerimeter() { return 2 * (w + h); }  
    @Override public double getArea() { return w * h; }  
    @Override public String toString() { return "w=" + w + ", h=" + h; }  
}
```

Interface – exemplo – demonstração

```
public class InterfaceDemo {  
    public static void main(String[] args) {  
        Measurable circle = new Circle(2.5);  
        Measurable rectangle = new Rectangle(4, 5);  
        System.out.printf("Perímetro círculo [%s]: %.2fcm\n", circle, circle.getPerimeter());  
        System.out.printf("Área círculo [%s]: %.2fcm2\n", circle, circle.getArea());  
        System.out.printf("Perímetro retângulo [%s]: %.2fcm\n", rectangle, rectangle.getPerimeter());  
        System.out.printf("Área retângulo [%s]: %.2fcm2\n", rectangle, rectangle.getArea());  
    }  
}
```

```
Perímetro círculo [r=2.5]: 15,71cm  
Área círculo [r=2.5]: 19,63cm2  
Perímetro retângulo [w=4.0, h=5.0]: 18,00cm  
Área retângulo [w=4.0, h=5.0]: 20,00cm2
```

Interface Comparable<T>

- Interface usada para impor uma ordem nos objetos que a implementam
- Pertence à *framework* de coleções Java
- Usada para ordenar elementos dentro de uma coleção
- Declara um único método

```
public int compareTo(T other);
```

 - Retorna um inteiro negativo, zero, ou um inteiro positivo função do objeto sobre o qual o método é chamado for inferior, igual ou superior a *other*
 - Tipo T: tipo genérico, traduzido por um tipo classe quando uma classe implementa esta interface

```
public class Xpto implements Comparable<Xpto> {  
    public int compareTo(Xpto x) { ... }  
}
```
- Impõe algumas regras de semântica:
 - Relação transitiva: se $A < B$ e $B < C$ então $A < C$
 - Desejável que se $a.compareTo(b)$ retornar 0, $a.equals(b)$ retorne true